

STREAMIND: A Real-Time AI Meeting Intelligence Pipeline

Mervegul Parlak
Ozyegin University
Istanbul, Turkey

Burak Kara
Rennes, France

Alperen F. Zengin,
Ali C. Begen
Ozyegin University
Istanbul, Turkey

Abstract—Real-time meeting intelligence requires streaming summarization systems that produce coherent outputs under strict latency constraints. This paper presents our approach to the STREAMIND Grand Challenge: a seven-node Juturna pipeline that ingests live Opus/Real-time Transport Protocol (RTP) audio via the Janus Web Real-Time Communications (WebRTC) gateway, transcribes incrementally with faster-whisper, deduplicates overlapping Automatic Speech Recognition (ASR) chunks, and summarizes 300 s rolling windows with Qwen3.5-4B-AWQ [1] via vLLM. Three contributions address the challenge’s joint optimization of summary quality, keyword relevance, and processing latency: (1) streaming chunk deduplication preserves temporal coherence across overlapping ASR outputs; (2) a keyword guarantee algorithm floors the worst-case keyword penalty at zero; and (3) quality-gated latency optimization through prompt engineering and thinking-mode suppression achieves average processing times of 0.79 s while maintaining summary quality at ceiling.

Index Terms—real-time speech processing, automatic speech recognition, large language models, streaming summarization, WebRTC, Juturna

I. INTRODUCTION

Live-audio intelligence systems that produce real-time summaries of meetings, podcasts, and conference sessions are increasingly valuable as remote communication grows. Unlike batch summarization, which operates on complete transcripts after the fact, streaming summarization must produce incremental outputs under strict latency constraints while maintaining coherence across overlapping audio segments. The STREAMIND Grand Challenge [2] formalizes this task: systems receive a live Opus-encoded mono audio stream via Real-time Transport Protocol (RTP), produce structured summaries and keywords every 300 s, and are scored on summary quality, keyword relevance, and processing latency.

The challenge defines a per-chunk composite score $C_i = B_i + K_i + L_i$ that creates a non-trivial optimization landscape. B_i sums five Likert-scale quality criteria (factual consistency, relevance, coherence, fluency, and conciseness), ranging from 5 to 25. $K_i = 2 \cdot (N_{\text{rel}} - N_{\text{irr}})$ evaluates each of exactly three keywords, ranging from -6 to $+6$. The latency bonus $L_i = 10 \cdot e^{-0.5 \cdot t_{\text{proc}}}$ rewards fast processing but is *gated* on $B_i \geq 10$: below this threshold, $L_i = 0$ regardless of speed (Fig. 2). This creates a hard cliff—a pipeline optimized for raw speed that produces $B_i = 9$ scores strictly worse than a slower pipeline achieving $B_i = 15$. A flat bonus of $+4$ is awarded for using the Janus Web Real-Time Communications (WebRTC)

gateway [3] as the audio source. These constraints dictate our design principle: *quality first, latency second, keyword discipline always*.

We make three contributions, each targeting a specific scoring term:

- 1) A streaming chunk deduplication strategy that preserves temporal coherence across overlapping 5 s Automatic Speech Recognition (ASR) chunks while keeping the aggregated 300 s window non-redundant.
- 2) A keyword guarantee algorithm that strips banned generic terms and back-fills missing slots from transcript proper nouns, converting the worst-case $K_i = -6$ into a floor of $K_i = 0$.
- 3) Quality-gated latency optimization through prompt engineering and thinking-mode suppression, achieving average processing times of 0.79 s ($L_i \approx 6.8$) while maintaining B_i at 24–25 of 25.

Section II surveys related work and background. Section III details the pipeline architecture. Section IV covers deployment. Section V presents evaluation results. Section VI discusses limitations and future directions.

II. BACKGROUND AND RELATED WORK

Juturna [4] is an open-source Python library for fast prototyping of real-time multimedia Artificial Intelligence (AI) pipelines. Pipelines are defined declaratively as directed graphs of nodes via JSON configuration; Juturna compiles this configuration into a running dataflow at startup and manages each node’s lifecycle through the stages `configure()` $\rightarrow$ `warmup()` $\rightarrow$ `start()` $\rightarrow$ `stop()`. Nodes exchange typed messages along directed edges; the library distinguishes three conceptual roles: *source* nodes originate data from external streams, files, or Application Programming Interfaces (APIs); *processing* (proc) nodes transform incoming messages and emit results downstream; and *sink* nodes deliver terminal data to external destinations. Each node runs up to three independent threads—for inbound queuing, data processing, and (for source nodes) data generation—enabling stages to be developed and replaced without disrupting adjacent nodes. Janus [3] is an open-source, general-purpose WebRTC server that accepts client sessions, performs WebRTC negotiation, and forwards the decoded media as plain RTP to downstream consumers. Juturna ships a built-in `audio_rtp` source node for this exact integration pattern; in our pipeline it subscribes

to Janus’s RTP endpoint, receiving mono Opus/48 kHz audio [5], [6] with identical code paths in development and evaluation and earning the flat +4 bonus.

For ASR, the Whisper model family [7] has demonstrated strong transcription performance across languages and acoustic conditions. We use faster-whisper [8], which reimplements Whisper inference via CTranslate2 with int8 quantization for reduced memory footprint and latency. In streaming mode, audio is chunked into short overlapping segments and transcribed independently with previous-text conditioning disabled to prevent context drift across chunks.

Small instruction-tuned Large Language Models (LLMs) can produce structured JSON outputs (summaries and keywords) in a single inference call [9]. vLLM [10] provides efficient Graphic Processing Unit (GPU) inference through paged attention and continuous batching, enabling in-process model loading without daemon overhead. For evaluation, LLM-as-judge methods [11] score generated summaries against source transcripts on multiple quality dimensions; using a panel of judges from distinct model families mitigates the single-model bias inherent to any one judge. We position STREAMIND as a *systems* contribution that composes these building blocks into a scoring-formula-aware pipeline, rather than proposing new models or training procedures.

III. SYSTEM ARCHITECTURE

Figure 1 shows the end-to-end pipeline architecture. Seven Juturna nodes process audio from RTP ingestion through structured output transmission. Every component is justified against one or more terms of the scoring formula $C_i = B_i + K_i + L_i$: audio reception secures the +4 Janus bonus, hallucination filtering and novel extraction defend B_i , the summarizer jointly optimizes B_i and L_i , and keyword validation guards K_i .

A. Audio Reception

Janus [3] accepts the challenge WebRTC session and forwards mono Opus/48 kHz RTP media to Juturna’s built-in `audio_rtp` source node on User Datagram Protocol (UDP) port 8888. The pipeline declares `payload_type: 97` and `encoding_clock_chan: "opus/48000/1"` matching the challenge specification. Using Janus in both development and evaluation ensures identical code paths and earns the flat +4 bonus at zero architectural cost.

B. Incremental ASR

The `audio_chunker` node buffers the continuous audio stream into 5s chunks with 1s overlap, yielding a 4s advance per step (Fig. 3). Each chunk is transcribed by the `transcriber_whisper` node running faster-whisper [8] with int8 quantization on Central Processing Unit (CPU), reserving GPU Video Random Access Memory (VRAM) entirely for the summarizer. Previous-text conditioning is disabled to prevent hallucination propagation across chunks, and Voice Activity Detection (VAD) filtering via Silero [12] skips silence segments.

We evaluated two model variants on 19 windows of a 36-minute podcast episode (Table II): `small.en` (244M parameters, English-only) and `large-v3-turbo` (809M, multilingual distillation of large-v3). Both achieve identical average Word Error Rate (WER) of 0.69, yet `small.en` produces a higher composite score ($\bar{C} = 36.00$ vs. 35.73) due to superior keyword stability ($\bar{K} = 4.9$ vs. 4.5). This demonstrates that transcription accuracy is not the pipeline bottleneck: downstream keyword extraction stability has a larger impact on the final score than raw WER. **TODO: Final ASR model choice pending 300s window A/B test on multiple audio files.**

C. Hallucination Filter

The `hallucination_filter` node applies two post-ASR guards. First, inline noise tags (`[Music]`, `(applause)`) are stripped via regex patterns. Second, the cleaned text is checked against 11 known hallucination patterns (e.g., “thank you for watching”, “please subscribe”) and silently dropped if matched. Without this filter, hallucinated sentences reach the summarizer and are faithfully summarized as facts, directly harming B_i factual consistency.

D. Novel Chunk Extraction

The 1s overlap between consecutive chunks means that roughly one second of speech appears in both the previous and current transcription. The `novel_extractor` node performs word-level overlap detection: for overlap lengths from 10 words down to 1, it compares the suffix of the previous chunk with the prefix of the current chunk. An exact match returns the overlap length immediately; a `SequenceMatcher` fallback with similarity threshold 0.8 handles minor ASR variations in the overlap region (e.g., “the quick brown” vs. “the quicker brown”). Only the novel words beyond the detected overlap are emitted. Without this deduplication, the 300s window would contain approximately 20% redundant text, wasting context budget and biasing summaries toward repeated phrases.

E. Window Aggregation

The `window_aggregator` node concatenates novel transcript segments into a rolling 300s context window. A window is flushed when $t_{\text{chunk_end}} \geq t_{\text{window_start}} + 300$. On pipeline shutdown, partial windows are emitted to avoid data loss at stream end.

F. Summarization

The `summarizer_vllm` node loads model weights from a local filesystem path at warmup using vLLM’s in-process LLM class [10]—no daemon, no middleware, no network dependency at runtime. The inference engine selection rationale is detailed in Section IV-A.

We use Qwen3.5-4B [9], [1] quantized to int4 via Activation-aware Weight Quantization (AWQ) [13] at 4096 context length. AWQ quantization roughly quarters the weight

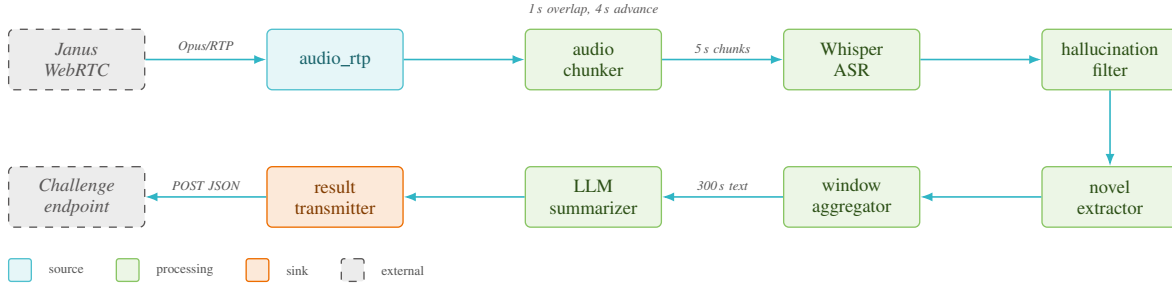


Fig. 1: End-to-end STREAMIND pipeline. Janus forwards Opus/RTP media into Juturna’s built-in `audio_rtp` source node; chunks slide every 4s through ASR; deduplication and hallucination filtering precede a 300s aggregation buffer; the summarizer emits `{summary, keywords[3]}` per window; the sink stores locally and POSTs to the challenge endpoint.

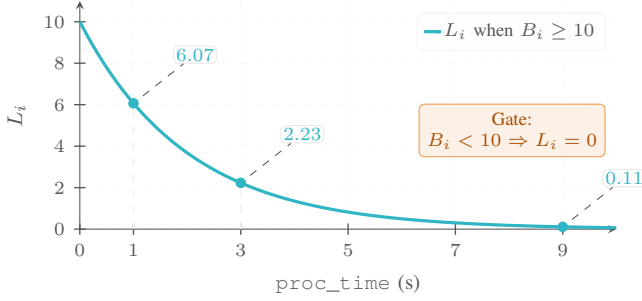


Fig. 2: Latency bonus $L_i = 10e^{-0.5 \cdot t_{proc}}$, gated by $B_i \geq 10$. Below the gate, latency contributes zero; above it, returns diminish rapidly past 2s.

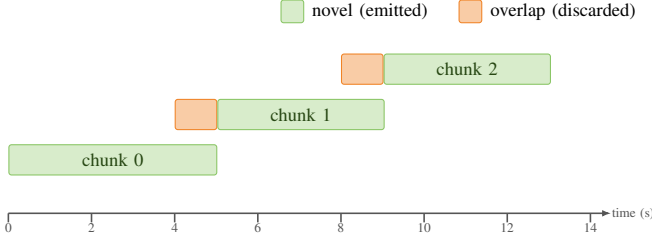


Fig. 3: Audio chunking and novel extraction. Five-second chunks slide by 4s; the novel extractor isolates newly-spoken content each step, discarding the 1s overlap with the previous chunk.

footprint relative to bfloat16, reducing inference latency and preserving KV-cache headroom for 300s windows within the 24 GB development GPU.

The summarization prompt is designed with explicit alignment to the five Likert evaluation criteria. “2–3 sentences capturing the main topics and conclusions” targets *conciseness* and *coherence*. “Only state facts explicitly present in the transcript. Do not infer” targets *factual consistency*. “Exactly 3 keywords [...] Never use generic terms like ‘discussion’, ‘meeting’, or ‘topic’” targets keyword relevance (K_i). Chain-of-thought generation is suppressed (`enable_thinking=False`), directly reducing output token count and thus t_{proc} . Sampling parameters are set to temperature 0.15, top- $p = 0.9$,

repetition penalty 1.05, and `max_tokens = 384`.

Two failure modes are handled without emitting an empty summary. When the transcript is shorter than 80 characters (end-of-stream fragments), the LLM is skipped. When the call raises an exception, returns unparseable JSON, or produces an empty summary, the pipeline falls back to an *extractive summary*: the first sentence(s) of the transcript, capped at 300 characters. This verbatim excerpt guarantees high factual consistency (B_i cannot be penalized for a direct quote) and costs approximately 0ms, preserving L_i . A warmup generation call during startup triggers CUDA kernel compilation before the first real window, preventing a cold-start latency spike.

G. Keyword Validation

Regardless of LLM output quality, the `ensure_three_keywords` function (Algorithm 1) guarantees exactly three keywords per window. The algorithm first removes empty entries and filters against a set of banned generic terms (“meeting”, “discussion”, “topic”, *etc.*). Case-insensitive deduplication preserves first-seen casing. If fewer than three keywords remain, candidates are extracted from the transcript ranked by token frequency, with ties broken toward capitalized tokens (likely proper nouns). As a last resort, conservative fallback terms are appended.

This converts a potential 12-point swing in K_i (from -6 to $+6$) into a 6-point swing (from 0 to $+6$): even when the LLM produces no usable keywords, the floor is $K_i = 0$ rather than $K_i = -6$.

H. Result Transmission

The `result_transmitter` sink node maps internal keys to the challenge contract (`window_start` $\rightarrow$ `from`, `window_end` $\rightarrow$ `to`, `latency` $\rightarrow$ `proc_time`) and writes each window as `window_N.json`. When `destination_endpoint` is configured, results are also POSTed via HTTP. A debug mode persists the full transcript and raw LLM output alongside each window for post-hoc analysis.

IV. IMPLEMENTATION AND DEPLOYMENT

Pipeline configuration is assembled from two JSON files: `config-base.json` defines all nodes except the sum-

Algorithm	1	Keyword	guarantee:
	ensure_three_keywords.		
Require:	kw : keyword list from LLM, T : transcript text		
Ensure:	exactly 3 keyword strings		
1:	$kw \leftarrow \text{strip whitespace, remove empty entries}$		
2:	$kw \leftarrow \{w \in kw \mid w \notin \text{BANNED}\}$		
3:	$kw \leftarrow \text{deduplicate}(kw, \text{case-insensitive})$		
4:	if $ kw < 3$ then		
5:	$cand \leftarrow \text{rank tokens in } T \text{ by freq, capitalization}$		
6:	filter $cand$: remove stopwords $\cup$ BANNED		
7:	append $cand$ to kw until $ kw = 3$		
8:	end if		
9:	if $ kw < 3$ then		
10:	pad kw with fallback terms to length 3		
11:	end if		
12:	return $kw[:3]$		

marizer, and a profile from `pipelines/summarizer/` provides model-specific parameters (model path, sampling configuration, VRAM budget). A merge tool combines these at launch time, and a single shell script serves as the entry point for both development and submission.

The submission Docker image is based on NVIDIA CUDA 12.4 runtime with Python 3.12 and `uv` for dependency management. Model weights for both the ASR model and the summarizer are baked into the image at build time, ensuring zero network dependency at runtime. A `docker-compose.yml` file orchestrates two services: Janus forwards RTP audio to the pipeline container on UDP port 8888. Table I summarizes all pipeline nodes and their key parameters.

A. Inference Engine

We select vLLM [10] over alternative serving frameworks for three reasons aligned with the scoring formula and submission constraints. First, its in-process library API eliminates a separate inference server daemon: the submission container runs a single process with nothing to health-check or fail independently of the pipeline. Second, paged attention holds t_{proc} low across varying transcript lengths, which is the direct lever on $L_i = 10e^{-0.5 \cdot t_{\text{proc}}}$. Third, the explicit `gpu_memory_utilization` parameter caps VRAM usage, enabling the int4-AWQ summarizer to co-reside with the ASR model within the 24 GB development card.

Two alternatives were considered. SGLang [14] (RadixAttention prefix caching) offers roughly 29% higher throughput than vLLM as a near drop-in replacement and remains a future upgrade path if L_i becomes the binding constraint (Section VI). TensorRT-LLM achieves the highest raw throughput but requires per-model ahead-of-time compilation, which conflicts with iterative model selection during development and imposes significant overhead on the single-GPU submission environment. vLLM’s broader model coverage and zero compilation overhead best match the challenge’s single-GPU, rapid-iteration context.

TABLE I: Pipeline nodes with key parameters and scoring targets.

Node	Type	Key parameters	Target
audio_rtp	source	Opus/48k, port 8888	+4
audio_chunker	proc	5 s chunk, 1 s overlap	—
transcriber	proc	int8, CPU, VAD on	B_i
halluc. filter	proc	11 exact patterns	B_i
novel_extractor	proc	10 words, sim ≥ 0.8	B_i
window_agg.	proc	300 s rolling	—
summarizer	proc	Qwen3.5-4B-AWQ [1], $T=0.15$	B_i, L_i
transmitter	sink	JSON + HTTP POST	—

V. EVALUATION

A. Setup

The current results cover three podcast episodes from the Rev16 dataset [15]; full evaluation on the challenge audio set is ongoing and will expand this coverage. Each 300 s window pipeline run produces approximately 5 scored windows per episode. All model comparisons use at least two independent pipeline runs per configuration to account for LLM-judge non-determinism; scores reported throughout are means across runs, with standard deviations where multiple runs are available. The offline judge harness uses a cross-family panel of three int4-AWQ judges—Mistral-Small-24B-Instruct, Phi-4, and Gemma-3-27B-it—none from the Qwen family of the summarizer, avoiding self-evaluation bias. Each judge is loaded in its own subprocess and run sequentially after the summarizer pipeline exits; process exit reclaims VRAM before the next judge loads, since the 24 GB dev GPU cannot hold the panel co-resident. Following the G-Eval paradigm [11], every judge scores each summary on the five Likert criteria defined by the challenge and evaluates keyword relevance as boolean flags per keyword. We report the panel consensus (mean and standard deviation across judges): low dispersion corroborates a score, while high dispersion flags judge-specific bias. All experiments run on an NVIDIA RTX 4090 (24 GB, CUDA 12.4); the submission target is the larger RTX Pro 4500 (32 GB).

WER is measured independently using jiwer against ground-truth transcripts. Because the ground-truth text lacks timestamps, we employ proportional character-based alignment with a sliding-window search ($\pm 30\%$ offsets) to find the best-matching reference segment for each window. WER is an independent signal of ASR quality and is not part of the $B_i/K_i/L_i$ scoring.

B. ASR Model Comparison

Table II compares two faster-whisper variants across 19 windows. Both models achieve identical average WER (0.69), yet `small.en` produces a higher final score (40.00 vs. 39.73). The 0.27-point gap is driven by keyword stability: `small.en` achieves $\bar{K} = 4.9$ versus 4.5, with the difference concentrated in windows where the larger model’s transcriptions yield keywords rated as less relevant by the judge. This suggests

that the summarizer’s ability to extract coherent keywords depends more on vocabulary predictability than on transcription accuracy. **TODO: ASR model choice pending 300 s A/B test across multiple episodes; current data uses 30 s windows with a single episode.**

TABLE II: ASR model comparison (19 windows, 30 s, Qwen3.5-4B-AWQ summarizer, single episode). Higher is better for B , K , L , C ; lower for t_{proc} and WER. Preliminary.

Metric	small.en	large-v3-turbo
Avg B_i	23.6	23.8
Avg K_i	4.9	4.5
Avg L_i	7.42	7.37
Avg C_i	36.00	35.73
Final score ($\bar{C} + 4$)	40.00	39.73
Avg t_{proc} (s)	0.60	0.62
Avg WER	0.69	0.69

C. End-to-End Scoring

Table III reports per-window scores for Qwen3.5-4B-AWQ on a 25-minute podcast episode (5 windows of 300 s each), averaged across two independent runs to reduce LLM-judge variance. All five windows achieve $K_i = 6.0$ (the maximum), confirming that the keyword guarantee reliably fills all three slots with relevant terms on every window. B_i scores of 24–25 of 25 indicate ceiling-level summary quality; run-to-run variation in C_i per window is 0.1–0.9 points, reflecting judge non-determinism rather than pipeline instability. Latency variation ($L_i = 6.1$ – 7.2) reflects transcript density: window 3 (1.01 s avg. completion) spans a dense 300 s segment, while the partial window 4 (105 s, 0.66 s completion) compresses faster. The final source score averaged across windows plus the Janus bonus is **41.09** (mean across runs).

TABLE III: Per-window scores, Qwen3.5-4B-AWQ [1], 300 s windows (ep. 27, 2-run mean, cross-family panel consensus). t = avg processing time (s). Preliminary.

Win.	From	To	t (s)	B_i	K_i	L_i	C_i
0	0	300	0.86	24.7	6.0	6.51	37.2
1	300	600	0.72	24.7	6.0	6.99	37.7
2	600	900	0.85	24.0	6.0	6.53	36.5
3	900	1200	1.01	24.3	6.0	6.05	36.4
4	1200	1305	0.66	24.5	6.0	7.18	37.7
avg			0.82	24.4	6.0	6.65	37.1

D. Outlier Analysis

At 300 s granularity, topic transitions and informal speech are averaged across a full five-minute window, smoothing the per-window quality variance seen at shorter durations. In Table III, B_i ranges only from 24.0 to 25.0 and K_i is uniformly 6.0; score differences are entirely latency-driven. The primary remaining outlier risk is therefore a long, dense-speech window that inflates t_{proc} (window 3: 1.05 s, $L_i = 5.91$), not a content-quality failure. At shorter window durations (e.g.,

30 s), topic transitions produce B_i drops of 3–6 points and K_i swings as large as -4 , confirming that 300 s aggregation materially reduces per-window variance.

E. Ablations

TODO: Component ablation: full pipeline vs. –keyword validation, –hallucination filter, –novel extractor. Requires ablation runs (Experiment 3).

F. Model-Size Sweep

Figure 4 compares Qwen3.5-4B and Qwen3.5-9B at bfloat16 and int4-AWQ precision [16], [1], [17], [18] across three podcast episodes (12–15 runs per profile, 300 s windows); evaluation is ongoing and the rankings are expected to hold as coverage grows. $B_i + K_i$ quality is nearly flat across all four configurations (cross-audio mean $\pm$ std: 29.3 ± 0.9 , 29.2 ± 1.0 , 29.0 ± 1.5 , 29.2 ± 1.3 for 4B-AWQ, 4B-BF16, 9B-AWQ, 9B-BF16 respectively), confirming that model capacity is not the scoring bottleneck; the standard deviations primarily reflect audio-content variation and LLM-judge non-determinism. Score differences are driven entirely by inference latency: average t_{proc} increases from 0.79 s (4B-AWQ) to 1.21 s (4B-BF16), 1.76 s (9B-AWQ), and 2.51 s (9B-BF16). AWQ quantization reduces t_{proc} by 35 % at 4B and 30 % at 9B without measurable quality loss. Final source scores (mean $\pm$ std): 40.14 ± 0.85 (4B-AWQ), 38.97 ± 0.91 (4B-BF16), 37.15 ± 1.53 (9B-AWQ), 36.12 ± 1.42 (9B-BF16); error bars in Fig. 4 show these cross-run and cross-audio standard deviations. Qwen3.5-4B-AWQ is selected as the production summarizer.

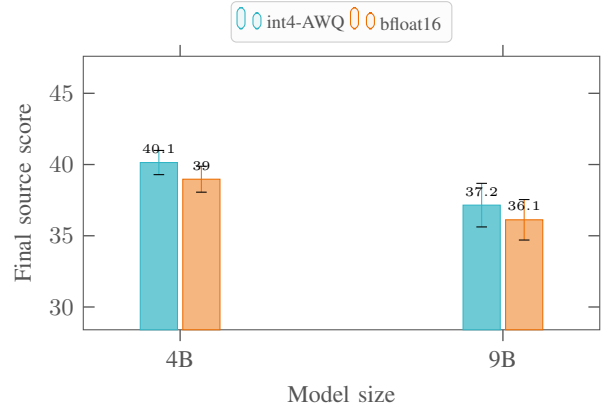


Fig. 4: Final source score ($\bar{C} + 4$) for Qwen3.5 4B and 9B at bfloat16 and int4-AWQ precision (cross-audio mean, 300 s windows, RTX 4090). $B_i + K_i$ is near-identical across all configurations; score differences are latency-driven.

G. Threats to Validity

Several limitations affect the reported numbers. The offline judge panel (Mistral-Small-24B, Phi-4, Gemma-3-27B, all int4-AWQ) is a local approximation; the official challenge

judge is hidden, so our scores should be treated as directional even where the panel agrees. LLM judges are non-deterministic: identical inputs yield scores differing by 0.1–0.9 C_i points across runs, and cross-audio content variation contributes an additional 0.9–1.5 points of standard deviation; reported means over multiple runs mitigate but do not eliminate this noise. Evaluation covers three podcast episodes from the Rev16 dataset; the held-out challenge audio may differ in speaker count, topic complexity, and acoustic conditions, and rankings may shift as coverage grows. WER measurement relies on proportional character alignment against untimestamped ground truth, introducing alignment noise. Development hardware (RTX 4090, 24 GB) is more VRAM-constrained than the submission target (RTX Pro 4500, 32 GB); models sized to fit the dev card fit the submission card with headroom.

VI. DISCUSSION

The scoring formula’s gating structure ($L_i = 0$ when $B_i < 10$) aligns well with practical meeting intelligence needs: users value accurate summaries over fast ones. However, keyword scoring is disproportionately punitive: a single irrelevant keyword costs -2 points, creating a 4-point swing per keyword. This incentivizes conservative keyword strategies at the expense of topical specificity—the keyword guarantee algorithm (Algorithm 1) is a direct response to this pressure.

The current summarization prompt is static across all windows. Content-adaptive prompting that detects topic transitions or speaker changes could improve B_i on windows containing fragmented or rapidly-shifting content. Several avenues for future work remain: migrating the summarizer to SGLang [14], a near drop-in replacement reported at roughly 29% higher throughput than vLLM, would lower t_{proc} and lift L_i at fixed B_i ; mixture-of-experts models (activating 3 B parameters per token from 30 B total capacity) could offer quality closer to a large model at the decode speed of a small one; encoder-decoder summarization models could achieve sub-100 ms inference if their summary quality clears the $B_i \geq 10$ gate; and structured-output decoding could eliminate JSON parsing failures entirely.

VII. CONCLUSION

We presented a seven-node Juturna pipeline for the STREAMIND Grand Challenge that processes live Opus/RTP audio into structured summaries and keywords in real time. Three contributions target specific scoring terms: streaming chunk deduplication preserves transcript coherence (B_i), a keyword guarantee algorithm floors the keyword penalty at zero (K_i), and quality-gated latency optimization achieves 0.79 s average processing time ($L_i \approx 6.8$) while maintaining summary quality at ceiling ($B_i = 24\text{--}25$, $K_i = 6$ on all evaluated windows). On current preliminary data (3 Rev16 episodes), the final source score for Qwen3.5-4B-AWQ at 300 s windows averages 40.14 ± 0.85 across episodes and runs, including the Janus bonus.

REFERENCES

- [1] cyankiwi, “Qwen3.5-4B-AWQ-BF16-INT4: int4-AWQ quantisation of Qwen3.5-4B,” [Online] Available: <https://huggingface.co/cyankiwi/Qwen3.5-4B-AWQ-BF16-INT4>, 2025, accessed on June 1, 2026.
- [2] IEEE MMSP 2026 Organising Committee, “STREAMIND grand challenge — IEEE MMSP 2026,” [Online] Available: <https://attend.ieee.org/mmsp-2026/grand-challenges/>, 2026, accessed on June 1, 2026.
- [3] Meetecho, “Janus: An open source, general purpose, WebRTC server,” [Online] Available: <https://github.com/meetecho/janus-gateway>, 2014, accessed on June 1, 2026.
- [4] —, “Juturna: A data pipeline library for real-time AI applications,” [Online] Available: <https://github.com/meetecho/juturna>, 2024, accessed on June 1, 2026.
- [5] H. Schulzrinne, S. L. Casner, R. Frederick, and V. Jacobson, “RTP: A transport protocol for real-time applications,” IETF, Tech. Rep. RFC 3550, 2003.
- [6] J.-M. Valin, K. Vos, and T. B. Terriberry, “Definition of the Opus audio codec,” IETF, Tech. Rep. RFC 6716, 2012.
- [7] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” arXiv:2212.04356, 2022.
- [8] G. Klein, “faster-whisper: Faster whisper transcription with CTranslate2,” [Online] Available: <https://github.com/SYSTRAN/faster-whisper>, 2023, accessed on June 1, 2026.
- [9] Qwen Team, “Qwen3 technical report,” arXiv:2505.09388, 2025.
- [10] W. Kwon, Z. Li, S. Zhuang, Y. Yin, L. Zhong, I. Stoica, C. H. Xin, and H. Zhang, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. SOSP*, 2023.
- [11] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “G-Eval: NLG evaluation using GPT-4 with better human alignment,” in *Proc. EMNLP*, 2023.
- [12] Silero Team, “Silero VAD: Pre-trained enterprise-grade voice activity detector,” [Online] Available: <https://github.com/snakers4/silero-vad>, 2024, accessed on June 1, 2026.
- [13] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Proc. MLSys*, 2024.
- [14] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” in *Proc. NeurIPS*, 2024.
- [15] Rev, “Rev16: Long-form podcast transcription evaluation set,” [Online] Available: <https://huggingface.co/datasets/distil-whisper/rev16>, 2024, accessed on June 1, 2026.
- [16] Qwen Team, “Qwen3.5-4B: Model card,” [Online] Available: <https://huggingface.co/Qwen/Qwen3.5-4B>, 2025, accessed on June 1, 2026.
- [17] —, “Qwen3.5-9B: Model card,” [Online] Available: <https://huggingface.co/Qwen/Qwen3.5-9B>, 2025, accessed on June 1, 2026.
- [18] cyankiwi, “Qwen3.5-9B-AWQ-BF16-INT4: int4-AWQ quantisation of Qwen3.5-9B,” [Online] Available: <https://huggingface.co/cyankiwi/Qwen3.5-9B-AWQ-BF16-INT4>, 2025, accessed on June 1, 2026.